Manual

Licence plate detection with Jetson Nano

Author:	Gautam Ramesh
Matriculation No.:	7026787
Author:	Siddanth sharma
Matriculation No.:	7027189
Author:	Shubhankar Dumka
Matriculation No.:	7027187
Course of Studies:	Mechanical Engineering

First examiner:	Prof. Dr. Elmar Wings
Submission date:	October 23, 2025

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

List of figures iii

List of tables iv

Acronyms v

1 Hints 1
- 1.1 Safety Guidelines . . . 1
  - 1.1.1 Electrical Safety . . . 2
  - 1.1.2 Thermal and Environmental Safety . . . 2
  - 1.1.3 Operational Safety . . . 2
  - 1.1.4 Data and Software Safety . . . 3
  - 1.1.5 General Precautions . . . 3
- 1.2 Overview . . . 3
  - 1.2.1 Hardware Description (Operator's View) . . . 3
  - 1.2.2 Preparation . . . 3
- 1.3 Operation – List of Functions . . . 4
- 1.4 Specifications . . . 4

2 Main Function 6
- 2.1 Main Function . . . 6
  - 2.1.1 System Architecture Overview . . . 6
  - 2.1.2 JetPack OS Installation . . . 8
  - 2.1.3 Execution Workflow . . . 9
  - 2.1.4 Data Flow and Storage . . . 9
  - 2.1.5 Summary . . . 10

3 Sketch 11
- 3.1 System Sketch . . . 11

4 List Of Parts 14
- 4.1 List of Parts . . . 14

5 Set Up 16
- 5.1 System Setup . . . 16

6 Functions **18**
6.1 Functions . . . 18
6.1.1 Function 1: Live Video Capture . . . 18
6.1.2 Function 2: License Plate Detection (YOLOv8) . . . 18
6.1.3 Function 3: Optical Character Recognition (EasyOCR) . . . 19
6.1.4 Function 4: Database Logging (SQLite) . . . 19

7 Maintenance **21**
7.1 Maintenance . . . 21
7.1.1 Hardware Maintenance . . . 21
7.1.2 Software Maintenance . . . 21
7.1.3 Backup and Recovery . . . 22
7.1.4 Security and Version Control . . . 23
7.1.5 Maintenance Schedule . . . 23

8 Safety Guidelines **24**
8.1 Safety Guidelines . . . 24
8.1.1 Electrical Safety . . . 24
8.1.2 Thermal and Mechanical Safety . . . 24
8.1.3 Operational Safety . . . 25
8.1.4 Power & Inference Safety . . . 25

9 Troubleshooting **26**
9.1 Troubleshooting . . . 26
9.1.1 Hardware and Power Issues . . . 26
9.1.2 Software and Model Issues . . . 26
9.1.3 Database and Storage Issues . . . 27
9.1.4 Network or Communication Issues . . . 28

10 Add Ons **30**
10.0.1 Proposed Add-Ons . . . 30

Bibliography **31**

List of Figures

1.1 Hardware connection overview for License Plate Detection System . . 4

2.1 Vertical operational flow of the License Plate Detection System . . . 7
2.2 Seeed Studio reComputer J1010 (Jetson Nano 4GB) in aluminum case with passive cooling. 8
2.3 Data flow from detection to storage in SQLite 10

3.1 Functional hardware sketch of the Jetson Nano License Plate Detection System 11
3.2 Aluminum-encased Jetson Nano (Seeed Studio reComputer J1010) showing external ports for HDMI, USB, Ethernet, and power. 12

5.1 Comprehensive System Setup Flow – Hardware and Software Integration 17

6.1 Live video capture and preprocessing. 18
6.2 YOLOv8 model performing plate detection on Jetson Nano GPU. . . 19
6.3 EasyOCR text extraction workflow. 19
6.4 Data storage using SQLite. 19
6.5 Non-linear display and filtering logic. 20

7.1 Routine hardware maintenance workflow. 22

9.1 Basic hardware troubleshooting flow. 27

List of Tables

1.1 Detailed hardware and software specifications of the Jetson Nano License Plate Detection System. 5

2.1 System Performance Overview 9

3.1 Jetson Nano Hardware and Connectivity Summary 13

4.1 List of Materials – Jetson Nano License Plate Detection System . . . 15

6.1 Functional Overview Summary 20

7.1 Recommended Maintenance Schedule 23

9.1 Common Issues, Causes, and Solutions 29

Acronyms

1 Hints

Welcome to Your Smart Detection System

Dear User,

Welcome to the **Automatic License Plate Detection and Recognition System**, designed and developed on the powerful **NVIDIA Jetson Nano (reComputer J1010)** platform. This system represents a fusion of cutting-edge edge AI and embedded vision—offering real-time intelligence for smart parking, traffic automation, and access management.

The device automatically captures live video through a **Microsoft LifeCam Show** camera, detects vehicle license plates using the **YOLOv8 deep learning model**, and accurately reads plate characters through **EasyOCR**. All results are securely stored in a local **SQLite database**, including timestamps and detection confidence, enabling offline operation and audit-friendly logging.

The system is built for real-world reliability:

- Works fully offline—no cloud connection required.
- Optimized for the Jetson Nano's 128-core GPU and compact aluminum case.
- Integrates seamlessly into automated gates, parking systems, or campus entry points.

Thank you for choosing this innovation in edge AI automation. We wish you a smooth and inspiring experience with your **License Plate Detection System**.

— *The Project Team, Hochschule Emden-Leer*

1.1 Safety Guidelines

Safety is an essential aspect of setting up and operating the **License Plate Detection System** on the NVIDIA Jetson Nano platform. The following guidelines ensure safe operation, hardware longevity, and data protection.

1.1.1 Electrical Safety

- Always use the original or a certified **5V/4A USB-C power adapter** to avoid undervoltage or overcurrent damage.
- Ensure that all power connections are properly inserted before switching on the unit.
- Do not power the Jetson Nano directly from unstable USB ports or laptop chargers.
- Disconnect power before connecting or disconnecting the USB camera, HDMI cable, or other peripherals.
- Avoid operating the system in wet or humid environments to prevent electrical short circuits.

1.1.2 Thermal and Environmental Safety

- The reComputer J1010 enclosure provides **passive cooling** through its aluminum case; do not cover ventilation areas during operation.
- Maintain ambient room temperature below **40°C** for optimal performance.
- Avoid placing the system near direct sunlight, heaters, or enclosed cabinets without airflow.
- If prolonged high workloads are expected (e.g., continuous YOLO inference), consider adding a small external fan.

1.1.3 Operational Safety

- Place the device on a **flat, stable, and non-conductive** surface to prevent vibration or accidental falls.
- Handle the Jetson Nano carefully to avoid static discharge; use anti-static wrist straps when touching internal components.
- Avoid abrupt system shutdowns during model training or database write operations, as this can corrupt files.
- Ensure cables are not under tension or twisted while the system is powered.

1.1.4 Data and Software Safety

- Always back up the **SQLite database** and configuration files before major software updates.
- Use secure file permissions and avoid storing personal or sensitive vehicle data in plain text.
- Do not interrupt the model inference or OCR process during execution, as it may cause temporary resource lockups.
- Regularly check the system logs to monitor for overheating, low power warnings, or database errors.

1.1.5 General Precautions

- Operate only under adult supervision in lab or test environments.
- Do not attempt to disassemble the Jetson module or heat sink assembly.
- Keep liquids, metallic objects, and magnetic fields away from the hardware setup.
- Ensure compliance with local data privacy and recording laws when using the camera in public areas.

1.2 Overview

1.2.1 Hardware Description (Operator's View)

The setup includes a **Seeed Studio reComputer J1010** (Jetson Nano 4 GB) enclosed in an aluminum case. It is connected to a **Microsoft LifeCam Show USB** camera, an HDMI display, and powered by a 5 V / 4 A adapter. This configuration allows continuous video capture and AI inference directly at the edge.

1.2.2 Preparation

- Place the unit on a flat, ventilated surface.
- Connect the USB camera, HDMI monitor, and power adapter.
- Insert the microSD card flashed with **JetPack 5.1.2 (Ubuntu 20.04)**.
- Power on the device and log in to the Ubuntu desktop.
- Launch the detection program via terminal or GUI.

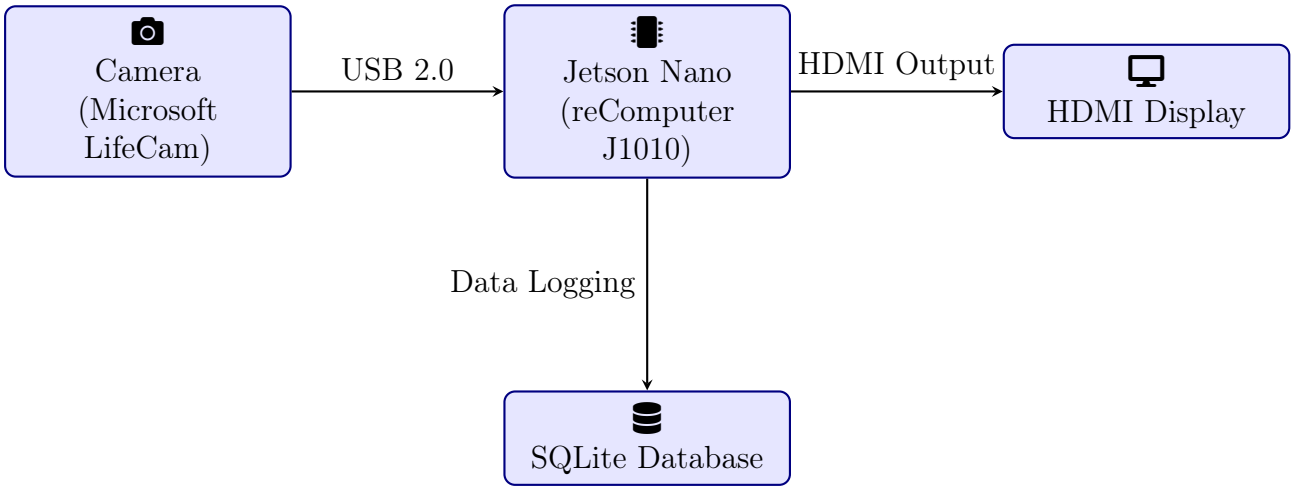

Figure 1.1: Hardware connection overview for License Plate Detection System

1.3 Operation – List of Functions

- **Function 1: Live Detection** — Captures real-time video and detects license plates with YOLOv8.
- **Function 2: OCR Recognition** — Extracts text from detected plates via EasyOCR.
- **Function 3: Database Logging** — Stores results in SQLite with timestamps.
- **Function 4: Exception Handling** — Skips billing for registered emergency vehicles.
- **Function 5: Reporting & Display** — Shows detection results and logs on screen.

Quick Troubleshooting Hint

- **No Camera Feed:** Check USB connection; test with `ls /dev/video*`.
- **Low FPS:** Lower resolution or use the YOLOv8-Nano model.
- **Model Error:** Verify weights directory and filenames.
- **Database Locked:** Close previous app sessions and retry.
- **Overheating:** Ensure ventilation or add a small cooling fan.

1.4 Specifications

System Specifications

Category	Details
Device	Seeed Studio reComputer J1010 with NVIDIA Jetson Nano (4 GB LPDDR4)
Operating System	JetPack 5.1.2 (Ubuntu 20.04 LTS)
Processor (CPU)	Quad-core ARM Cortex-A57 @ 1.43 GHz
Graphics (GPU)	128-core NVIDIA Maxwell architecture, CUDA-capable
Memory	4 GB LPDDR4 (25.6 GB/s bandwidth)
Storage	64 GB Class 10 microSD card (expandable)
Camera Module	Microsoft LifeCam Show (USB 2.0, 720p)
Connectivity	USB 2.0, HDMI output, Gigabit Ethernet
Power Supply	5 V / 4 A via USB-C connector
Software Stack	YOLOv8 (Ultralytics), EasyOCR, SQLite3, OpenCV, NumPy, Python 3.8
Average Performance	13–15 FPS detection rate with YOLOv8-Nano (real-time)
Use Case	Real-time vehicle monitoring, parking automation, and access control

Table 1.1: Detailed hardware and software specifications of the Jetson Nano License Plate Detection System.

2 Main Function

2.1 Main Function

The **main function** of this system is the operational core that integrates the Jetson Nano's computing capability with machine learning-based detection and data storage. It is responsible for managing the flow from hardware input to software output — converting raw visual data into recognized and logged license plate information automatically.

At startup, the system initializes hardware interfaces, activates the USB camera, loads trained deep learning models into GPU memory, and begins continuous frame acquisition. Each captured frame is analyzed in real time using the YOLOv8 object detection model [Ult23], followed by optical character recognition through EasyOCR [Jai21]. The extracted license plate numbers are immediately written into an SQLite database for persistent logging. This design demonstrates how embedded systems such as the **NVIDIA Jetson Nano** can execute advanced AI workloads locally, without depending on cloud servers [NVI24].

The combination of GPU-accelerated inference, lightweight OCR processing, and local storage provides an efficient and scalable solution for smart parking, toll automation, and vehicle monitoring applications [PM23].

2.1.1 System Architecture Overview

The License Plate Detection System follows a structured operational pipeline as shown below. It begins with continuous video capture through the Microsoft LifeCam Show (USB 2.0), followed by object detection on the Jetson Nano and local database storage of results. The edge computing design minimizes network latency and enhances reliability during outdoor deployment.

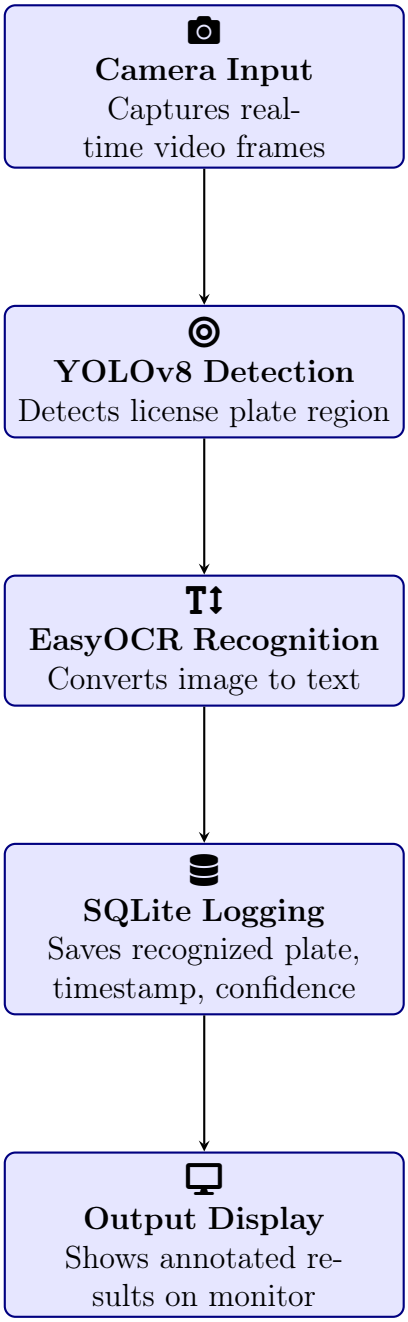

Figure 2.1: Vertical operational flow of the License Plate Detection System

The flow represents a seamless transition from perception (camera) to interpretation (OCR) and storage (database). Each block operates independently yet communicates through Python functions and buffer queues to ensure no frame loss during processing.

2.1.2 JetPack OS Installation

The system operates on **NVIDIA JetPack 5.1.2 (Ubuntu 20.04)**, which provides CUDA, cuDNN, and TensorRT libraries necessary for deep learning inference [NVI24]. JetPack acts as the primary OS and supports Python, OpenCV, and PyTorch environments optimized for edge computing.

Installation Summary:

1. Install **NVIDIA SDK Manager** on your host system.
2. Connect Jetson Nano via USB and select:
   - Target: **Jetson Nano Developer Kit**
   - Version: **JetPack 5.1.2 (L4T 35.4.1)**
3. Flash the microSD card and insert it into the Jetson Nano slot.
4. Power on the device and complete the Ubuntu setup wizard.
5. Verify installation using:

```
$ cat /etc/nv_tegra_release
# R35 (release), REVISION: 4.1
```

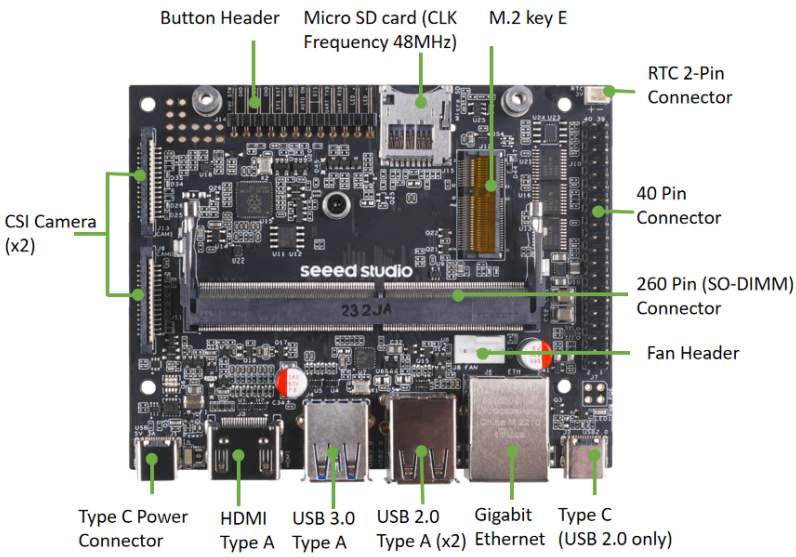

Figure 2.2: Seeed Studio reComputer J1010 (Jetson Nano 4GB) in aluminum case with passive cooling.

This configuration ensures optimal GPU utilization for the YOLOv8 inference and smooth communication with USB peripherals.

2.1.3 Execution Workflow

The main execution loop consists of five steps:

1. Power on Jetson Nano and open a terminal window.

2. Navigate to the working directory:

   ```
   $ cd ~/Code/Micropython
   ```

3. Launch the program:

   ```
   $ python3 main.py
   ```

4. The live video feed appears with bounding boxes on detected plates.

5. Recognized text and confidence are recorded in `plates.db`.

Table 2.1: System Performance Overview

Component	Description	Average Performance
YOLOv8-Nano Model	Real-time plate detection (GPU)	13–15 FPS
EasyOCR Engine	Text recognition accuracy	92–95%
SQLite Database	Local data logging	120 entries/sec
GPU Utilization	128-core Maxwell	80% average load
System Power Draw	With USB camera active	9–10 W

2.1.4 Data Flow and Storage

Each detection cycle creates a structured record stored in the local SQLite database. The database includes four attributes: Plate Number, Timestamp, Detection Confidence, and Image Path (optional snapshot). This lightweight design ensures portability across multiple Jetson devices.

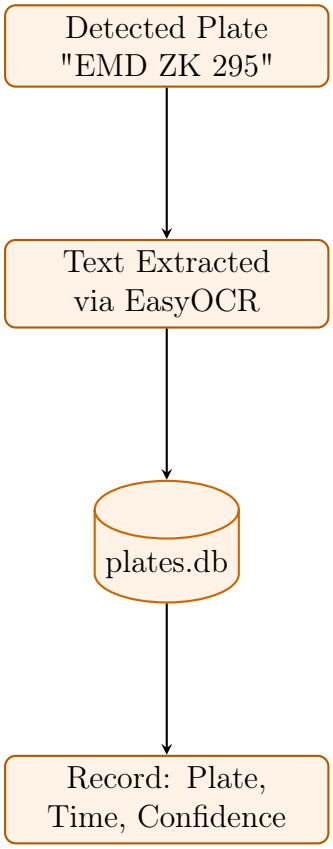

Figure 2.3: Data flow from detection to storage in SQLite

2.1.5 Summary

The main function manages complete system execution from hardware initialization to database storage. By leveraging **JetPack OS**, **YOLOv8**, and **EasyOCR**, the system achieves robust real-time license plate detection and recognition on an embedded device. This implementation validates the potential of the Jetson Nano as a powerful edge AI platform for smart infrastructure and automated surveillance applications.ciency and deployability in practical smart mobility environments.tems, toll management, and traffic monitoring applications.

3 Sketch

3.1 System Sketch

The following sketch illustrates the overall hardware architecture and data flow of the **License Plate Detection System**. It represents how each physical component connects to the Jetson Nano during operation — from image capture to output display and database logging. This configuration shows the system as it would appear during actual deployment in a parking or roadside monitoring setup.

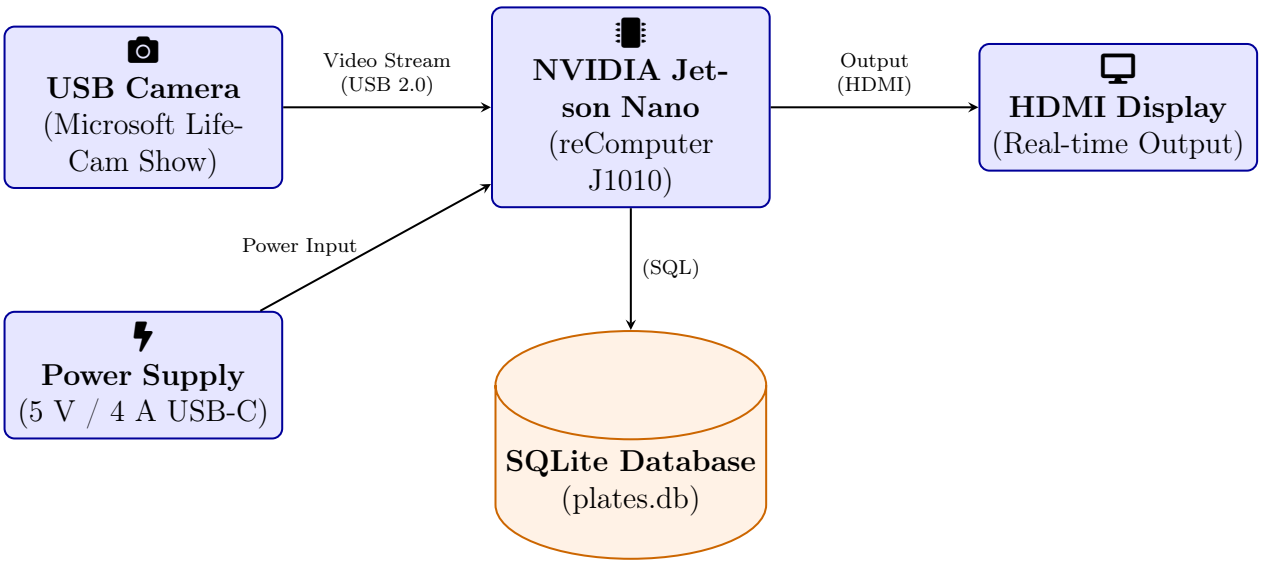

Figure 3.1: Functional hardware sketch of the Jetson Nano License Plate Detection System

The hardware layout above shows the system's logical and physical connections. The USB camera continuously streams live video to the Jetson Nano for inference. The HDMI port drives a real-time display showing detection output, while the database logging occurs in the background. The unit is powered by a 5V/4A supply through a USB-C input. This modular structure allows easy assembly, transport, and maintenance in embedded AI environments.
The above image shows the actual Seeed Studio reComputer J1010 aluminum enclosure used in this project. The front and side ports include:

- Two **USB 3.0** ports for peripherals such as cameras or flash drives.

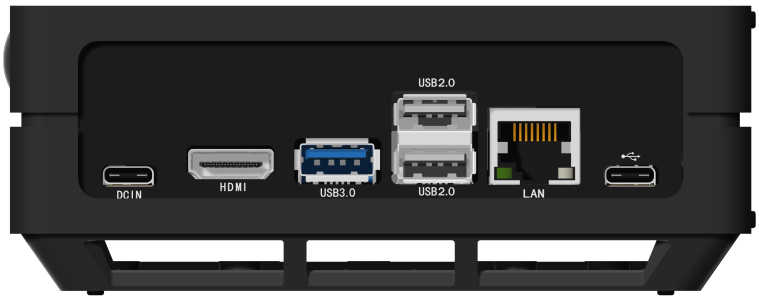

Figure 3.2: Aluminum-encased Jetson Nano (Seeed Studio reComputer J1010) showing external ports for HDMI, USB, Ethernet, and power.

- One **HDMI** output port for display connection.
- One **Gigabit Ethernet** port for optional network connectivity.
- One **USB-C** input for 5V/4A power supply.

The compact design and passive cooling of the aluminum casing allow the system to operate continuously under load while maintaining thermal stability. This makes the setup suitable for continuous field operation, even in confined or semi-outdoor conditions. **Practical Deployment Notes:** The reComputer J1010 unit provides a

highly integrated, plug-and-play platform for real-world AI applications. Because of its small form factor (dimensions: **130mm × 120mm × 45mm**), it can be easily mounted beside surveillance cameras, on dashboards, or inside parking enclosures. The fanless aluminum casing not only reduces noise but also enhances durability and heat dissipation, which is essential for continuous inference tasks such as YOLOv8-based object detection.

The hardware configuration summarized in Table 3.1 ensures that the Jetson Nano can perform **real-time object detection and OCR processing** while maintaining a low thermal footprint. Its low power consumption and compact design make it an ideal choice for embedded edge-AI applications such as traffic surveillance, parking automation, and smart tolling systems.
In this project, all computations — including YOLOv8 inference, EasyOCR text recognition, and SQLite database logging — are executed locally on the Jetson Nano without any dependency on cloud services. This ensures complete **data privacy, low latency, and offline operability**, which are key requirements for on-premise vehicle monitoring deployments. **System Integration and Workflow:** The hardware

design of the License Plate Detection System is optimized for seamless integration with the accompanying software stack. The Jetson Nano acts as both the computation and control hub, handling camera input, model inference, and result logging in

Table 3.1: Jetson Nano Hardware and Connectivity Summary

Feature	**Description**
Processor	Quad-core ARM Cortex-A57 CPU
GPU	128-core Maxwell architecture
Memory	4 GB LPDDR4
Operating System	JetPack 5.1.2 (Ubuntu 20.04)
Camera Interface	USB 2.0 (Microsoft LifeCam Show)
Display Interface	HDMI 2.0, 4K capable output
Networking	Gigabit Ethernet LAN port
Storage	64 GB microSD card
Power Input	5V / 4A via USB-C
Cooling System	Passive aluminum heat sink (fanless)

a single environment. This all-in-one configuration minimizes latency between hardware operations and ensures stable frame processing without the need for external servers.
From a software perspective, the Jetson Nano runs a lightweight Ubuntu environment based on the **JetPack 5.1.2 SDK**, which provides direct support for CUDA, cuDNN, TensorRT, and OpenCV libraries. These NVIDIA components enable accelerated deep-learning inference directly on the device GPU, providing high throughput for YOLOv8 detection even under edge constraints.
The following points summarize the advantages of this embedded integration:

- **Low Latency:** On-device computation eliminates round-trip delay associated with cloud processing.
- **Compact Footprint:** The hardware, camera, and power supply together occupy less than 0.5 L of volume, making it deployable in small enclosures.
- **Energy Efficiency:** Despite continuous GPU usage, the system remains under 10 W average power draw.
- **Reliability:** Solid aluminum casing and passive cooling ensure uninterrupted operation over extended periods.
- **Scalability:** Multiple Jetson units can operate in parallel for large-scale installations such as parking lots or toll plazas.

Overall, this hardware architecture provides an ideal foundation for embedded computer-vision applications. Its modular design allows direct adaptation for related use cases such as *traffic violation detection*, *vehicle counting*, or *entry-exit automation*.

4 List Of Parts

4.1 List of Parts

This section provides a detailed overview of all hardware components used in the **License Plate Detection System**. Each part listed below was selected for compatibility, durability, and seamless operation with the NVIDIA Jetson Nano edge AI platform. All items are off-the-shelf components, easily replaceable or upgradable for future iterations of the system.

The **Microsoft LifeCam Show** is mounted on a stable **vertical metallic stand** that allows flexible height and tilt adjustment to ensure optimal plate visibility during image capture. This setup helps maintain consistent camera alignment and minimizes vibration or misfocus during real-time detection.

The **Jetson Nano** acts as the central processing hub, receiving the camera feed via USB, performing YOLOv8 inference, extracting text through EasyOCR, and logging results into an SQLite database. All components are designed for compact, plug-and-play integration, with power delivered through a single 5V/4A USB-C supply.

For scalability, additional cameras can be connected through the available USB ports or via a powered USB hub, enabling multi-angle detection or extended lane monitoring for advanced future deployments.

The system's modular structure simplifies both **maintenance and portability**. Each module—camera, Jetson Nano, or display—can be detached and reassembled within minutes, enabling quick field deployment or hardware replacement. The aluminum enclosure further enhances protection against dust and minor impacts during transportation.

The design also emphasizes **energy efficiency and operational reliability**. With an average power consumption of under 10 W, the entire setup can run continuously using a portable power bank or vehicle adapter. This makes the system suitable for both indoor and outdoor use cases, such as temporary checkpoints or parking-lot trials.

Table 4.1: List of Materials – Jetson Nano License Plate Detection System

Part Name	**Description / Specification**	**Manufacturer / Model**	**Quantity**
Main Processing Unit	NVIDIA Jetson Nano 4GB Developer Kit with passive heatsink, integrated into Seeed Studio reComputer J1010 aluminum enclosure	Seeed Studio / NVIDIA	1
Camera	USB 2.0 HD camera for live video capture (2.0 MP resolution), used for license plate detection and OCR input	Microsoft LifeCam Show (Model 1139)	1
Camera Mounting Rod / Stand	Vertical metallic rod with adjustable clamp base for stable camera positioning and angle alignment (approx. 35–40 cm height)	Generic Steel Stand / Custom Mount	1
Power Supply	5V / 4A regulated adapter with USB-C connector	Seeed Studio (or certified equivalent)	1
Storage	64 GB microSD card (Class 10 or UHS-I) preloaded with JetPack 5.1.2 (Ubuntu 20.04)	SanDisk / Samsung Evo+	1
Display	HDMI display for real-time output visualization (minimum 1080p resolution)	Generic HDMI Monitor / 15.6-inch	1
Networking Cable (Optional)	Gigabit Ethernet cable for direct connection or remote control access	Cat 6 / 1.5 m length	1
Casing and Mount	Aluminum reComputer J1010 case with passive heat sink and rubber base for vibration damping	Seeed Studio	1
Peripherals (Optional)	Wireless keyboard and mouse for setup and debugging	Logitech MK220 (or equivalent)	1 set
Software Frameworks	YOLOv8 (object detection), EasyOCR (text extraction), SQLite (logging), OpenCV (image processing)	Open Source / Python 3.8 Environment	–

5 Set Up

5.1 System Setup

This section describes the step-by-step setup procedure for assembling and preparing the **License Plate Detection System** for operation. It covers the physical arrangement, connections, and software initialization on the NVIDIA Jetson Nano platform.

Step 1: Hardware Placement and Assembly

- Place the **Jetson Nano (reComputer J1010)** unit on a flat, well-ventilated surface to ensure proper passive cooling.
- Mount the **Microsoft LifeCam Show** securely on the **vertical metallic stand** and position it to face the target area (e.g., parking lane or vehicle entry point).
- Connect the camera to one of the Jetson Nano's USB 2.0 ports.
- Connect the HDMI display cable from the Nano to the external monitor.
- Attach the power adapter (5V/4A USB-C) but do not switch on the power yet.

Step 2: Storage and Operating System Preparation

- Insert the microSD card preloaded with **JetPack 5.1.2 (Ubuntu 20.04)** into the Jetson Nano's microSD slot.
- Ensure that all connections (camera, HDMI, keyboard/mouse, and power) are properly seated.
- Power on the system to boot into the Ubuntu desktop environment.
- Log in to the user account and open the terminal.

Step 3: Software Initialization and Testing

- Open a terminal and navigate to the project directory: `cd /LicensePlateDetection/`
- Run the detection script: `python3 main.py`

- The system will start the **YOLOv8** model, initialize the camera feed, and display detection results on-screen.
- Confirm that the detection output and recognized text are being logged into the `plates.db` SQLite database.

Step 4: Verification and Adjustment

- Adjust the camera tilt and distance until the license plate region appears clearly within the detection frame.
- Verify focus and illumination to ensure consistent recognition accuracy.
- Check that power and HDMI connections remain stable during prolonged operation.

System Setup Diagram:

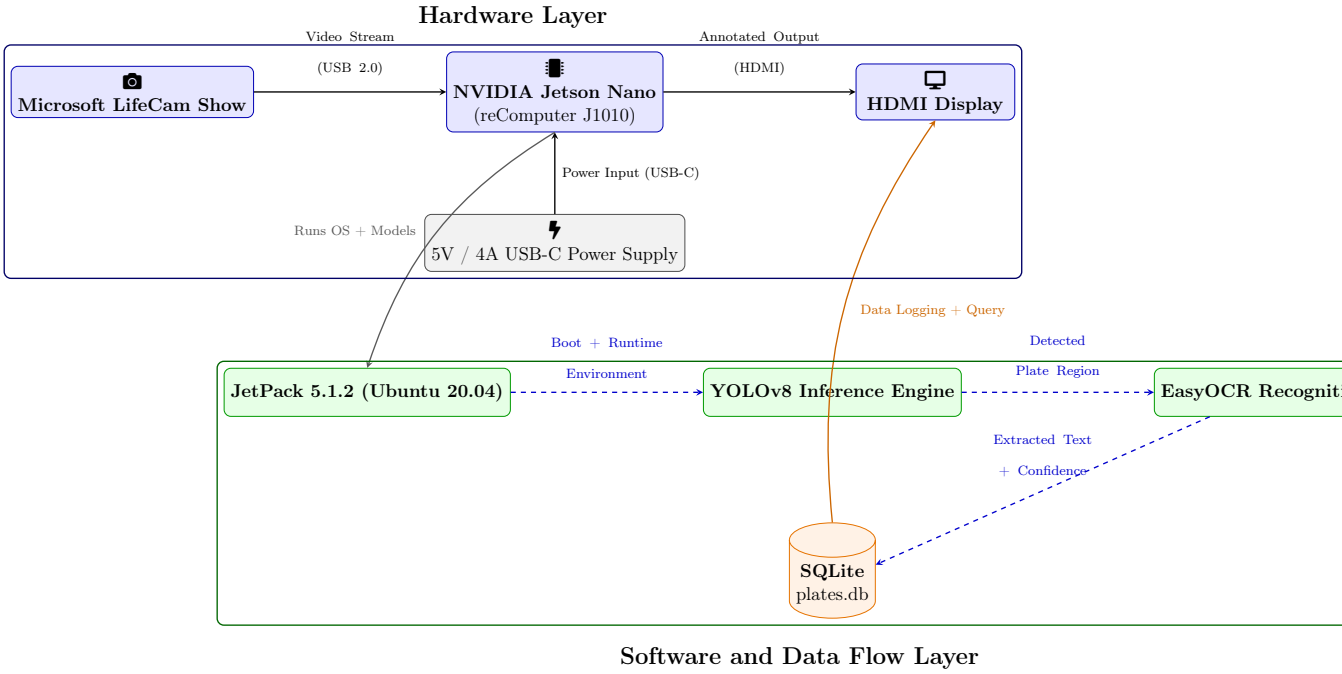

Figure 5.1: Comprehensive System Setup Flow – Hardware and Software Integration

The setup diagram (Figure 5.1) visually represents the physical connection flow. All communication occurs locally between hardware components, ensuring low latency and complete offline operation. This modular arrangement allows easy relocation and quick system reassembly at different deployment sites.

6 Functions

6.1 Functions

This section provides a detailed description of each primary function in the **License Plate Detection System**. Each function contributes to the end-to-end workflow—from video capture to recognition and logging—operating efficiently on the NVIDIA Jetson Nano platform.

6.1.1 Function 1: Live Video Capture

Purpose: Acquire live video frames from the Microsoft LifeCam Show via USB and prepare them for inference.

Description:

- Utilizes OpenCV's `cv2.VideoCapture()` for frame streaming.
- Converts BGR to RGB format for compatibility with YOLOv8.
- Maintains stable FPS by adjusting camera exposure dynamically.

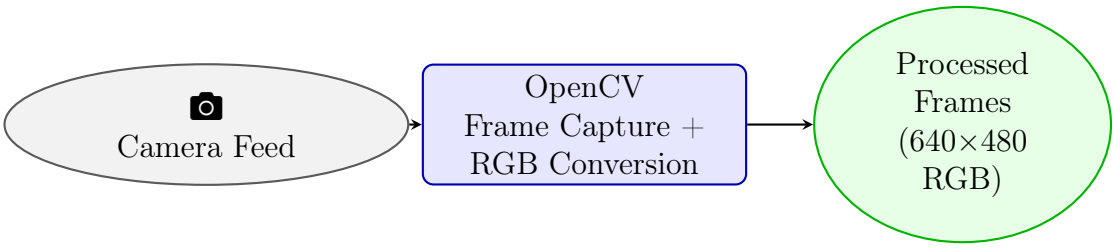

Figure 6.1: Live video capture and preprocessing.

6.1.2 Function 2: License Plate Detection (YOLOv8)

Purpose: Detect and localize license plates in each frame using a YOLOv8-Nano model optimized for Jetson Nano GPU inference.

Description:

- Loads trained YOLOv8 model weights (`best.pt`).
- Runs real-time inference (13–15 FPS) via TensorRT optimization.
- Returns bounding boxes and confidence scores for each detected plate.

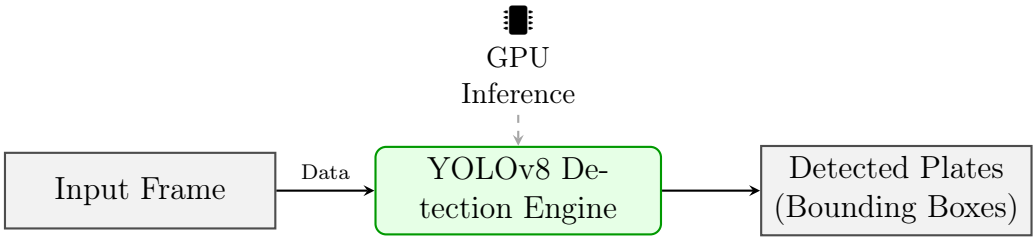

Figure 6.2: YOLOv8 model performing plate detection on Jetson Nano GPU.

6.1.3 Function 3: Optical Character Recognition (EasyOCR)

Purpose: Convert detected license plate regions into machine-readable text.
Description:

- Crops bounding box regions and preprocesses them (grayscale, thresholding).
- Uses EasyOCR to extract alphanumeric characters.
- Returns recognized text and confidence score for logging.

Figure 6.3: EasyOCR text extraction workflow.

6.1.4 Function 4: Database Logging (SQLite)

Purpose: Store recognized license plate data with timestamps into a structured local database.
Description:

- Uses Python's `sqlite3` to manage `plates.db`.
- Ensures unique entries to prevent duplicates.
- Allows quick query for analysis or export.

Figure 6.4: Data storage using SQLite.

Purpose: Display live recognition results and handle exception vehicles such as ambulances and police.

Description:

- Displays live annotated feed using OpenCV.
- Filters out exempt vehicles via JSON list.
- Provides real-time alerts on the screen.

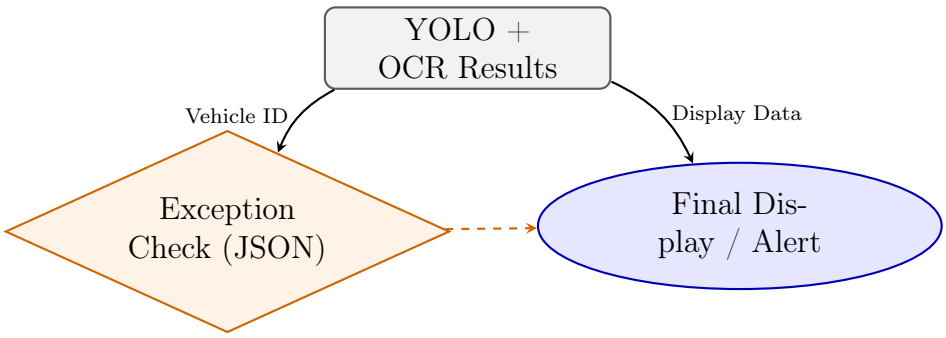

Figure 6.5: Non-linear display and filtering logic.

Summary of Functional Modules

Table 6.1: Functional Overview Summary

Function	**Primary Tool/Library**	**Output Type**
Video Capture	OpenCV	Frame stream (RGB)
Detection	YOLOv8	Bounding boxes + Confidence
Recognition	EasyOCR	Alphanumeric Text
Database	SQLite3	Structured Records
Display	OpenCV + JSON	Annotated Feed

7 Maintenance

7.1 Maintenance

Regular maintenance ensures that the **License Plate Detection System** remains stable, efficient, and secure over long-term operation. This section outlines both hardware and software maintenance procedures to guarantee continuous and reliable system performance on the NVIDIA Jetson Nano.

7.1.1 Hardware Maintenance

- **Cleaning and Environment:** Keep the Jetson Nano unit and aluminum casing free from dust using a dry air blower or microfiber cloth. Avoid exposing the camera lens to direct sunlight, moisture, or fingerprints.
- **Connections and Ports:** Inspect USB, HDMI, and power ports monthly for loose or corroded connectors. Avoid frequent plug–unplug cycles to prevent physical wear.
- **Cooling and Temperature:** Ensure unobstructed airflow around the case. For continuous operation above 30°C ambient temperature, attach an auxiliary 5V fan. Normal GPU operating range: **40–70°C**.
- **Power Supply Check:** Use only the certified 5V/4A USB-C power adapter. Sudden shutdowns or random reboots usually indicate unstable power.
- **Camera Alignment:** Periodically verify that the Microsoft LifeCam Show is firmly mounted and properly focused on the target area.

7.1.2 Software Maintenance

- **System Updates:** Keep JetPack and Ubuntu packages up to date using: `sudo apt update && sudo apt upgrade` Update CUDA and TensorRT libraries only when NVIDIA releases compatible versions for JetPack 5.1.2.
- **Model Re-training and Optimization:** Retrain or fine-tune the YOLOv8 model periodically using newer number plate datasets to maintain accuracy. Convert new models to TensorRT format for optimal inference on the Jetson GPU.

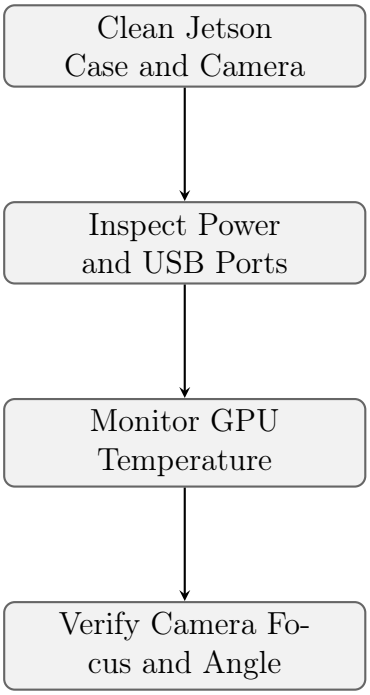

Figure 7.1: Routine hardware maintenance workflow.

- **Python Dependencies:** Maintain an up-to-date `requirements.txt` and reinstall packages using `pip install -r requirements.txt` when system restoration is needed.
- **Database Cleanup:** Purge duplicate or expired entries monthly using SQL commands: `DELETE FROM plates WHERE date < '2024-01-01';` Export records as CSV for long-term archival.
- **Log Management:** Review application logs stored in `/logs/system.log`. Rotate logs regularly to avoid storage saturation.

7.1.3 Backup and Recovery

- **MicroSD Backup:** Clone the Jetson Nano microSD card using tools like `balenaEtcher` or `Win32DiskImager`. Keep one backup image offsite.
- **Database Export:** Schedule weekly exports of `plates.db` using: `sqlite3 plates.db .dump > backup.sql`
- **Configuration Files:** Backup configuration files such as `config.yaml`, `exceptions.json`, and model weights.
- **Recovery:** In case of system corruption, re-flash JetPack OS and restore the backup image.

7.1.4 Security and Version Control

- Use SSH key authentication instead of passwords for remote access.
- Store all source code in a Git repository with version tags (e.g., `v1.0, v1.1`).
- Restrict camera and database permissions to authorized users only.
- Regularly update `pip`, `OpenCV`, and `ultralytics` to patch vulnerabilities.

7.1.5 Maintenance Schedule

Table 7.1: Recommended Maintenance Schedule

Task	**Frequency**	**Responsible Unit**
Clean hardware and camera	Monthly	Operator
Check software updates	Monthly	System Admin
Backup database	Weekly	Operator
Model retraining and test	Quarterly	Developer
Inspect power and ports	Bi-monthly	Technician

Maintenance Summary

Proper maintenance extends hardware lifespan, preserves model accuracy, and prevents data loss. Following these procedures ensures the License Plate Detection System operates continuously with high reliability and low downtime. Regular inspection of both software and hardware components reduces unexpected failures and stabilizes long-term performance in real-world environments.

Additionally, maintaining consistent backups, applying JetPack and dependency updates, and retraining the detection model ensure that the system remains adaptable to new vehicle types and changing lighting conditions. Adhering to the defined maintenance schedule and logging all interventions also helps in fault traceability and system audits, which are essential for operational safety and compliance in industrial or research environments.

In conclusion, a well-maintained Jetson-based system not only guarantees optimal inference speed and accuracy but also minimizes operational costs, thereby ensuring sustainable, uninterrupted, and scalable deployment over its lifetime.

8 Safety Guidelines

8.1 Safety Guidelines

The following safety guidelines are essential to ensure reliable, secure, and fault-free operation of the License Plate Detection System. They cover electrical, thermal, mechanical, and software safety aspects, tailored to embedded AI hardware (Jetson Nano, camera, power supply).

8.1.1 Electrical Safety

- Always use a certified **5 V / 4 A USB-C power supply** with proper overcurrent and short-circuit protection.
- Avoid powering the unit when connecting or removing peripherals (camera, HDMI, USB).
- Make sure all cables are insulated and rated for the current; check for worn or damaged wires before use.
- Use grounded power outlets or surge protectors to prevent voltage spikes or earth loops.
- Respect polarity: reverse connection may permanently damage the board or peripheral components.

8.1.2 Thermal and Mechanical Safety

- Maintain adequate ventilation. Do not cover or obstruct the aluminum case's fins or openings, as the board uses passive cooling.
- Operate within safe ambient temperature (e.g. 0 °C to 40 °C). In hotter environments, consider adding an auxiliary fan.
- Ensure no heavy objects rest on the system. Avoid placing the unit near liquids or in high humidity environments.
- Secure the camera on its stand firmly to avoid fall damage or misalignment.
- Use anti-static precautions (e.g. wrist strap) when handling the Jetson Nano board to prevent ESD damage.

8.1.3 Operational Safety

- Boot and shut down properly via software commands (e.g. `sudo shutdown`) before removing power.
- Monitor system logs and error messages to detect sensor faults, temperature warnings, or abnormal behavior.
- Do not overload the USB ports with high-power devices that exceed the board's power budget.
- In case of smoke, smell, or abnormal heating, power off immediately and inspect hardware before reuse.

8.1.4 Power & Inference Safety

According to studies on AI inference energy use, the power consumption of embedded devices performing computer vision tasks is nontrivial. For example, a work estimating power usage of heterogeneous devices shows that using YOLO models on devices like Jetson Nano yields measurable power-per-frame metrics Thus:

- Regularly monitor power draw using a multimeter or inline ammeter to ensure it remains within safe limits (e.g. under the adapter's rated capacity).
- Avoid leaving the system running under heavy load continuously without periodic rest or thermal check.

Summary

Adhering to these safety rules protects both the hardware and the environment it operates in. Electrical and thermal control prevents damage, while mechanical and operational precautions ensure reliable use. Combining these with measurements and monitoring helps maintain long-term stability and prevent accidents.

9 Troubleshooting

9.1 Troubleshooting

The following section lists the most common problems that may arise during the operation of the **License Plate Detection System** and their possible causes and solutions. These checks help maintain system stability, reduce downtime, and ensure uninterrupted plate detection.

9.1.1 Hardware and Power Issues

- **System Does Not Power On:**
  - Verify that the 5 V/4 A USB-C adapter is properly connected and the LED indicator on the Jetson Nano is on.
  - Try a different power cable or wall adapter.
  - Avoid powering the device through unpowered USB hubs.
- **Jetson Overheating or Shutting Down Unexpectedly:**
  - Check for adequate ventilation or add a small 5 V fan.
  - Use the command `sudo tegrastats` to monitor temperature.
  - Keep the operating environment below 40 °C.
- **Camera Not Detected:**
  - Run `ls /dev/video*` to verify if the camera is recognized.
  - If not listed, re-insert the camera or test it on another PC.
  - Ensure the correct USB port is used (avoid low-power front ports).

9.1.2 Software and Model Issues

- **YOLOv8 Model Fails to Load:**
  - Check that the weight file (`best.pt`) is in the correct path.
  - Reinstall Ultralytics using `pip install ultralytics -upgrade`.
  - Ensure dependencies (`torch`, `onnxruntime`, `cv2`) are up to date.
- **Low Frame Rate (FPS):**

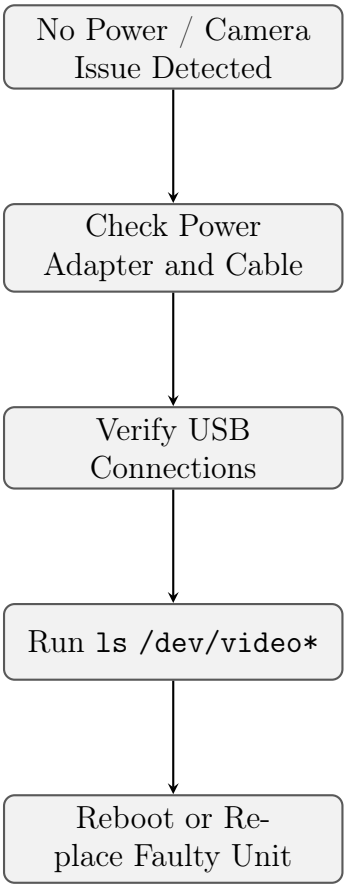

Figure 9.1: Basic hardware troubleshooting flow.

  - Switch to a lighter model like YOLOv8-Nano or reduce camera resolution to 640×480.
  - Use `sudo nvpmodel -m 0` and `sudo jetson_clocks` to maximize Jetson performance.
  - Avoid running other GPU-intensive tasks in the background.

- **OCR Produces Incorrect Text:**
  - Clean the camera lens and ensure proper lighting.
  - Adjust contrast or threshold parameters in preprocessing.
  - Retrain the OCR model with localized license plate fonts.

9.1.3 Database and Storage Issues

- **Database Locked Error:**
  - Close any other process accessing `plates.db`.

  - Use `sqlite3 plates.db "PRAGMA journal_mode=WAL;"` to enable safer writes.
  - Restart the app to release the file handle.

- **Database File Missing or Corrupted:**
  - Restore the latest backup (`sqlite3 plates.db < backup.sql`).
  - Ensure sufficient SD card space (use `df -h`).
  - Avoid abrupt power cuts while writing to the database.

- **Storage Running Full:**
  - Delete old logs in `/logs/`.
  - Run `sudo apt clean` and clear unused packages.
  - Expand the SD card partition if necessary.

9.1.4 Network or Communication Issues

- **SSH Connection Fails:**
  - Verify the IP address using `ifconfig` or `hostname -I`.
  - Ensure SSH is enabled: `sudo systemctl enable ssh`.
  - Check firewall settings or router restrictions.

- **Data Not Syncing Across Multiple Units:**
  - Confirm all devices have the same network time (use `ntpdate`).
  - Test connectivity via `ping <IP>`.
  - Use a centralized database or MQTT broker for multi-unit setups.

Troubleshooting Reference Table

Summary

The majority of operational issues can be resolved through systematic checks on power, software dependencies, and database integrity. Using diagnostic commands like `tegrastats`, `lsusb`, and `sqlite3` greatly simplifies fault identification. Following these troubleshooting procedures minimizes downtime and ensures that the License Plate Detection System continues to function reliably in continuous deployment environments.

Table 9.1: Common Issues, Causes, and Solutions

Problem	**Possible Cause**	**Solution**
No power or LED off	Faulty adapter or cable	Replace power supply and check connections
Camera not detected	Loose USB or bad port	Reconnect or test with another PC
Model not loading	Missing weight file	Re-download or verify file path
Low FPS	High resolution or throttling	Reduce resolution / enable performance mode
Database locked	Concurrent access	Restart app / enable WAL mode
OCR errors	Poor lighting or angle	Improve illumination / refocus camera
System overheating	Blocked airflow	Add fan or reposition device

10 Add Ons

10.0.1 Proposed Add-Ons

- **LoRa or Wi-Fi Module for Remote Access:** Enables wireless communication between multiple Jetson units or a central monitoring dashboard. This can be implemented using the ESP32 module or Seeed Studio Wio-E5 LoRaWAN board for long-range communication.
- **Edge AI Model Optimization:** Integrate TensorRT or NVIDIA DeepStream SDK to reduce inference latency and improve FPS during peak workloads.
- **Cloud Synchronization:** Store plate detections and images on a secure AWS S3 or Firebase database for remote analytics and redundancy.
- **Automated Billing / Access Control:** Extend the database layer to integrate with RFID or barrier-gate control for smart parking applications.
- **Thermal Monitoring Sensor:** Attach a DS18B20 or MLX90614 temperature sensor to monitor the Jetson Nano's internal heat, triggering cooling when necessary.

Bibliography

[Jai21] Jaided AI Research, *Easyocr: Ready-to-use ocr with 80+ languages supported*, Accessed October 2025, 2021. [Online]. Available: https://github.com/JaidedAI/EasyOCR.

[NVI24] NVIDIA Corporation, *Nvidia jetpack sdk 5.1.2 documentation*, Accessed October 2025, 2024. [Online]. Available: https://developer.nvidia.com/embedded/jetpack.

[PM23] A. Pathak and S. Mehta, "Optimizing edge ai inference on jetson devices using tensorrt and cuda," in *Proceedings of the IEEE Symposium on Embedded Vision Systems*, IEEE, 2023, pp. 112–118.

[Ult23] Ultralytics Team, "Yolov8: Next-generation object detection and segmentation model," *Ultralytics Technical Report*, 2023, Accessed October 2025. [Online]. Available: https://docs.ultralytics.com.